

# pamica: GPU-accelerated Adaptive Mixture Independent Component Analysis in Python with Fortran parity

Seyed Yahya Shirazi<sup>1</sup>, Arnaud Delorme<sup>1,2</sup>, and Scott Makeig<sup>1</sup>

<sup>1</sup> Swartz Center for Computational Neuroscience, Institute for Neural Computation, University of California San Diego, USA <sup>2</sup> Centre de Recherche Cerveau et Cognition (CerCo), CNRS, University of Toulouse, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

## Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

## Summary

Independent Component Analysis (ICA) is a standard method for separating electroencephalography (EEG) and electromyography (EMG) recordings into maximally independent sources, which isolates brain, muscle, and artifact activity for downstream analysis. Adaptive Mixture ICA (AMICA) generalizes single-model ICA to a mixture of ICA models with adaptive source densities, and produces the most dipolar (and thus most physiologically interpretable) component decompositions of EEG among the widely used algorithms benchmarked by Delorme et al. (2012). Its reference implementation is a Fortran program parallelized with the Message Passing Interface (MPI) and distributed as a compiled binary driven from MATLAB/EEGLAB, which is difficult to install, runs only on the CPU, and is not usable from a Python scientific workflow.

pamica is a Python implementation of AMICA that reproduces the reference Fortran results within numerical tolerance while running on the CPU, NVIDIA GPUs (CUDA), and Apple GPUs (Apple's MLX array framework (Hannun et al., 2023)). It is a complete NumPy/PyTorch reimplementation, not a wrapper around the Fortran binary, built on PyTorch (Paszke et al., 2019), NumPy (Harris et al., 2020), and SciPy (Virtanen et al., 2020) (Python \$ \$3.12, PyTorch \$ \$2.12; tested on Apple Silicon/ARM64 and x86\_64/CUDA Linux), and exposes a scikit-learn-style estimator under a BSD-3-Clause license. In double precision it reproduces the reference score-function algebra to machine precision; it also runs in single precision (float32), which the CPU-only binary does not offer and which is numerically faithful (agreeing with double precision to four to five significant digits) yet required to use Apple GPUs, which have no float64 and host the fastest backend (MLX). pamica writes output in the binary format that EEGLAB's AMICA loader reads: a single-model output file is byte-identical in layout to a native AMICA file and needs no manual re-interpretation, and multi-model output round-trips through the same loader. Correctness is defined as parity with the Fortran reference for the single-model case and, because multi-model AMICA is not partition-identifiable, as a similar distribution of solutions for the multi-model case; both are validated on real EEG against the reference binary. A minimal example is a scikit-learn-style call, `AMICA(n_models=1, n_mix=3).fit(X)` on a (channels, samples) array (full workflow in the README). The software is at <https://github.com/scen/pyAMICA> (archived at doi:10.5281/zenodo.21312148).

## Statement of need

AMICA yields components that are well suited to equivalent-dipole source localization and to automated classification of independent components (Pion-Tonachini et al., 2019), and it separates EEG more effectively than most alternatives (Delorme et al., 2012; Palmer et al.,

2012). The reference Fortran implementation shares these limitations and is invoked as an external process from MATLAB rather than called from Python. As neuroimaging analysis has moved toward Python, for example MNE-Python (Gramfort et al., 2013), an AMICA that runs natively in Python and on a GPU, and that is validated to reproduce the Fortran reference numerically, is needed for modern pipelines and for studying the algorithm in an open codebase.

General-purpose Python ICA implementations do not fill this gap. scikit-learn and MNE-Python provide FastICA (Hyvärinen & Oja, 2000) and Infomax (Bell & Sejnowski, 1995; Lee et al., 1999), and Picard (Ablin et al., 2018) provides a faster maximum-likelihood ICA, but none implement AMICA's mixture of models, adaptive generalized-Gaussian source densities, or Newton updates, and so they do not reproduce AMICA decompositions. pamica targets researchers who need AMICA: EEG/EMG analysts who want AMICA-quality decompositions inside a Python pipeline, users of GPU hardware who want faster runs than the CPU-only binary, and methodologists who need a transparent reference implementation to inspect and build on.

## Implementation and validation

pamica provides a natural-gradient (Amari, 1998) expectation-maximization backend that ports the full AMICA algorithm: exact-EM mixture updates, a positive-definite Newton step (Palmer et al., 2008), symmetric zero-phase-component-analysis (ZCA) sphering, the five source-density families of the reference (generalized Gaussian, Gaussian, logistic, sub-Gaussian, and the extended-Infomax kurtosis switcher), a mixture of ICA models, and component sharing across models.

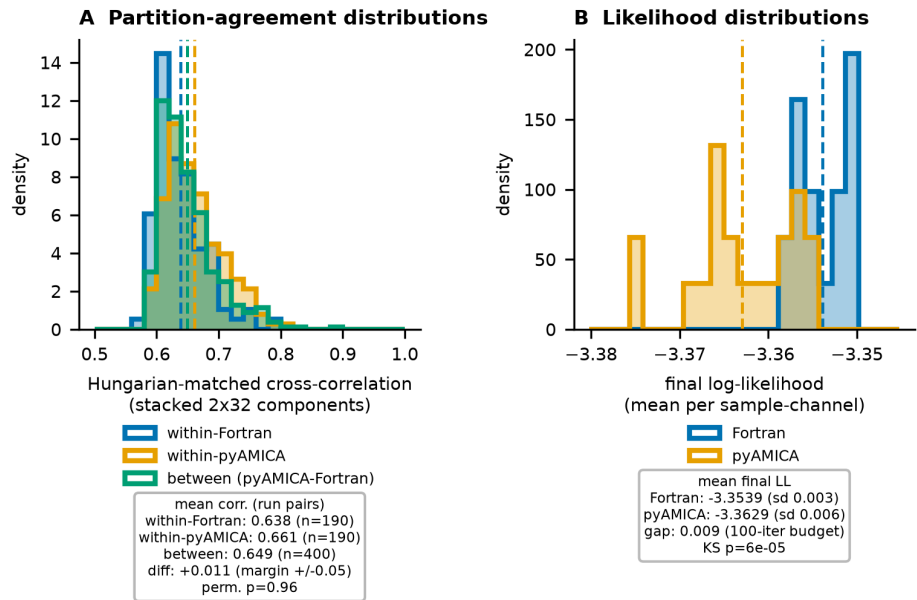
pamica's conformity with the reference binary is measured with two complementary metrics: Hungarian-matched component correlation, and the Amari distance (Amari et al., 1996), a standard unmixing-matrix comparison metric that needs no assignment step since it is permutation- and scale-invariant by construction. Both implementations were run for AMICA's usual 2000 iterations with Newton off (`do_newton=0`) and otherwise-default parameters (two separate configuration files drive them, JSON for pamica and Fortran's own text format, with a transcribed subset of settings; a shared-format reader is planned). The single-model headline (Table 1) uses an external recording (OpenNeuro ds002718, 70 channels,  $k \approx 153$ ), well past the  $\sim 60$  threshold where cross-backend agreement plateaus (documentation); the bundled 32-channel sample used below sits at that threshold's boundary ( $k \approx 30$ ) and gives a consistent Amari distance. Score functions and per-block sufficient statistics are exact to floating-point resolution against the literal Fortran expressions on the bundled sample. A mixture of ICA models is not partition-identifiable, so exact partition parity is the wrong bar for the multi-model case; it is instead assessed by whether the two implementations sample a similar distribution of solutions, across ensembles of 20 bundled-sample runs each (Figure 1). A run-level permutation test, which permutes the 40 runs as intact units to respect the dependence among pairwise values, finds no evidence that cross-implementation agreement is worse than Fortran's own run-to-run agreement, so single-run values reflect intrinsic estimator spread rather than a shortfall. The multi-model log-likelihood distributions still differ slightly at a matched 100-iteration budget (pamica reaches Fortran's mean with about twice as many iterations), so full-likelihood similarity is not yet claimed. Per-run detail for both metrics is in the documentation.

**Table 1:** Single-model parity (external ds002718,  $k \approx 153$ ; Amari and score-function checks on the bundled sample) and multi-model distributional similarity (bundled sample) of pamica with the Fortran reference. All values are means (sd shown where relevant) over matched components (single-model) or 190 within- and 400 cross-implementation pairs (multi-model). Full methodology and reproduction steps are in the documentation.

| Regime       | Metric                                                                                          | Result (mean; correlation / Amari distance)                      |
|--------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Single-model | Log-likelihood gap to Fortran (mean per-sample-channel LL, $-3.6993$ , $k \approx 153$ )        | within $\sim 0.0005$                                             |
| Single-model | Conformity with Fortran                                                                         | $0.998$ ( $k \approx 153$ ) / $0.006$ (bundled, $k \approx 30$ ) |
| Single-model | Score functions (non-default families) and sufficient statistics                                | exact to floating-point resolution ( $\sim 10^{-15}$ )           |
| Multi-model  | Single-run magnitude (pamica-Fortran; Fortran-Fortran)                                          | $0.65$ ; $0.64$ (sd $0.05$ ) / $0.163$ ; $0.174$ (sd $0.02$ )    |
| Multi-model  | Ensemble similarity, 20 runs each: mean difference, between — within-Fortran (permutation $p$ ) | $+0.011$ ( $p = 0.96$ ) / $-0.011$ ( $p > 0.999$ )               |

#### Multi-model AMICA ( $n\_models=2$ ): pyAMICA vs Fortran ensembles, real sample EEG

Dashed vertical lines mark each distribution's mean.



**Figure 1:** Multi-model solution-ensemble partition-correlation distributions (panel A) and log-likelihood distributions (panel B) for 20 pamica and 20 Fortran fits of the sample EEG; dashed lines mark each distribution's mean. The within-Fortran, within-pamica, and between-implementation correlation distributions overlap, so the single-run correlation reflects the estimator's intrinsic run-to-run spread rather than a gap to the reference. Panel B's apparent separation is a  $0.009$  log-likelihood gap on a  $\sim 0.035$  axis.

85 All backends converge to the same single-model log-likelihood on real EEG (maximum pairwise  
86 difference  $\sim 0.003$ ). On Apple Silicon the MLX backend is the fastest option and is flat with  
87 channel count (15-25 ms per iteration from 16 to 70 channels; see the documentation), about  
88 eight times faster than double-precision multithreaded CPU; PyTorch-MPS is not a win (at or  
89 worse than the CPU). On NVIDIA hardware double-precision CUDA is the reproducible path  
90 and is overhead-bound at EEG scale, so single precision gives it little additional speedup (Table  
91 2). Native Fortran itself scales with CPU cores, unlike the CPU backends above: with enough  
92 cores pinned it beats the CUDA GPU on the workstation (Table 2), though only by dedicating  
93 most cores of a larger, hotter host than a laptop GPU; it does not catch Apple's MLX on  
94 laptop hardware. A data-size sweep further shows cross-backend component agreement rising  
95 with frames per channel and plateauing near 0.98 once the decomposition is well-determined,  
96 where two independent double-precision implementations (native Fortran and PyTorch-CUDA)  
97 agree at a mean of 0.995; single-precision runs agree with double precision to four to five  
98 significant digits, so double precision remains the default for parity.

**Table 2:** Single-model throughput on real 70-channel EEG ( $n_{\text{mix}}=3$ ,  $\text{pdftype}=0$ ,  $\text{block\_size}=512$ ; warm, minimum of repeated runs). CPU, MPS, and MLX on Apple Silicon; CUDA on a separate NVIDIA RTX 4090; CUDA float32 is comparable ( $\sim 36$  ms). The two native-Fortran rows are from a separate core-count sweep (documentation) on the same two machines, at each backend's throughput plateau; the other CPU rows above use the platform default thread count, so they are not core-matched to Fortran. Unlike the correctness comparison, this benchmark uses external data (OpenNeuro ds002718, one subject so far) and specific GPU hardware.

| Backend (device)                                | Precision | ms / iteration |
|-------------------------------------------------|-----------|----------------|
| MLX (Apple GPU)                                 | float32   | 25             |
| CUDA (NVIDIA RTX 4090)                          | float64   | 39             |
| Native Fortran (Intel Core i9-13900K, 24 cores) | float64   | 30             |
| PyTorch CPU (Apple Silicon)                     | float64   | 193            |
| Native Fortran (Apple Silicon, 8 cores)         | float64   | 70             |
| PyTorch MPS (Apple GPU)                         | float32   | 255            |
| NumPy (reference, Apple Silicon)                | float64   | 622            |

99 The correctness harness compares pamica against Fortran with two metrics, Hungarian-matched  
100 component correlation and Amari distance, and never uses synthetic data; the multi-model and  
101 score-function checks need no external download (bundled sample only). The full per-channel  
102 and multi-model performance tables, the per-run Amari-distance detail, and the data-size  
103 sweep, along with the step-by-step commands to reproduce every number here, are in the  
104 documentation (<https://eeglab.org/pyAMICA/guides/validation/>).

## 105 State of the field

106 pamica complements rather than replaces EEGLAB (Delorme & Makeig, 2004) and its Fortran  
107 AMICA plugin: it reads and writes the same output format, so results move between the two,  
108 while adding GPU support and a Python API. Two other Python AMICA reimplementations  
109 have appeared (Esmaeili, 2025; Herforth, 2026), both of which provide MNE-Python-compatible  
110 objects; pamica instead offers a scikit-learn-style array API and byte-identical EEGLAB I/O, and  
111 does not yet ship an MNE-Python wrapper. What distinguishes pamica is the rigor and scope  
112 of its Fortran-parity validation, beyond what either alternative publishes: source-density score  
113 functions exact to floating-point resolution ( $\sim 10^{-15}$ ), single-model component correlation  
114 and Amari distance against the reference, and a distributional-similarity framework for the  
115 non-identifiable multi-model case, together with byte-identical EEGLAB output and an MLX  
116 backend for Apple GPUs.

## Acknowledgements

We thank Jason Palmer and Ken Kreutz-Delgado, co-developers of AMICA, for the reference implementation, and the EEGLAB community for the tools and sample data used to validate this work. Two of the authors are original developers of the methods *pamica* builds on: S.M. co-developed the AMICA algorithm (Palmer et al., cited above) and A.D. is a lead developer of EEGLAB (Delorme & Makeig, 2004). This work was supported by The Swartz Foundation (Old Field, NY) to the Swartz Center for Computational Neuroscience and by National Institutes of Health grant R01-NS047293 (to A.D. and S.M.).

## References

- Ablin, P., Cardoso, J.-F., & Gramfort, A. (2018). Faster independent component analysis by preconditioning with hessian approximations. *IEEE Transactions on Signal Processing*, 66(15), 4040–4053. <https://doi.org/10.1109/TSP.2018.2844203>
- Amari, S.-I. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276. <https://doi.org/10.1162/089976698300017746>
- Amari, S.-I., Cichocki, A., & Yang, H. H. (1996). A new learning algorithm for blind signal separation. *Advances in Neural Information Processing Systems*, 8, 757–763.
- Bell, A. J., & Sejnowski, T. J. (1995). An information-maximization approach to blind separation and blind deconvolution. *Neural Computation*, 7(6), 1129–1159. <https://doi.org/10.1162/neco.1995.7.6.1129>
- Delorme, A., & Makeig, S. (2004). EEGLAB: An open source toolbox for analysis of single-trial EEG dynamics including independent component analysis. *Journal of Neuroscience Methods*, 134(1), 9–21. <https://doi.org/10.1016/j.jneumeth.2003.10.009>
- Delorme, A., Palmer, J., Onton, J., Oostenveld, R., & Makeig, S. (2012). Independent EEG sources are dipolar. *PLoS ONE*, 7(2), e30135. <https://doi.org/10.1371/journal.pone.0030135>
- Esmaeili, S. (2025). *Amica: Native python implementation of AMICA for MNE-Python and scientific EEG workflows*. <https://github.com/snesmaeili/amica>.
- Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267. <https://doi.org/10.3389/fnins.2013.00267>
- Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible machine learning on apple silicon*. <https://github.com/ml-explore/mlx>.
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Herforth, J. (2026). *Pyamica: PyTorch AMICA, adaptive mixture independent component analysis*. <https://github.com/DerAndereJohannes/pyamica>.
- Hyvärinen, A., & Oja, E. (2000). Independent component analysis: Algorithms and applications. *Neural Networks*, 13(4-5), 411–430. [https://doi.org/10.1016/S0893-6080\(00\)00026-5](https://doi.org/10.1016/S0893-6080(00)00026-5)
- Lee, T.-W., Girolami, M., & Sejnowski, T. J. (1999). Independent component analysis using an extended infomax algorithm for mixed subgaussian and supergaussian sources. *Neural Computation*, 11(2), 417–441. <https://doi.org/10.1162/089976699300016719>
- Palmer, J. A., Kreutz-Delgado, K., & Makeig, S. (2012). *AMICA: An adaptive*

- 161 *mixture of independent component analyzers with shared components.* Swartz  
162 Center for Computational Neuroscience, University of California San Diego.  
163 [https://sccn.ucsd.edu/~jason/amica\\_a.pdf](https://sccn.ucsd.edu/~jason/amica_a.pdf)
- 164 Palmer, J. A., Kreutz-Delgado, K., Rao, B. D., & Makeig, S. (2008). Newton method for the  
165 ICA mixture model. *2008 IEEE International Conference on Acoustics, Speech and Signal*  
166 *Processing (ICASSP)*, 1805–1808. <https://doi.org/10.1109/ICASSP.2008.4517982>
- 167 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,  
168 Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-  
169 performance deep learning library. *Advances in Neural Information Processing Systems*, 32,  
170 8024–8035.
- 171 Pion-Tonachini, L., Kreutz-Delgado, K., & Makeig, S. (2019). ICLabel: An automated  
172 electroencephalographic independent component classifier, dataset, and website.  
173 *NeuroImage*, 198, 181–197. <https://doi.org/10.1016/j.neuroimage.2019.05.026>
- 174 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,  
175 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:  
176 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3),  
177 261–272. <https://doi.org/10.1038/s41592-019-0686-2>